

the problem has $\Theta(n)$ inherent parallelism. Unfortunately, simple fork-join parallelism does not let you achieve this maximal parallelism.

26-6 *Randomized parallel algorithms*

Like serial algorithms, parallel algorithms can employ random-number generators. This problem explores how to adapt the measures of work, span, and parallelism to handle the expected behavior of randomized task-parallel algorithms. It also asks you to design and analyze a parallel algorithm for randomized quicksort.

- a. Explain how to modify the work law (26.2), span law (26.3), and greedy scheduler bound (26.4) to work with expectations when T_P , T_1 , and T_∞ are all random variables.
- b. Consider a randomized parallel algorithm for which 1% of the time, $T_1 = 10^4$ and $T_{10,000} = 1$, but for the remaining 99% of the time, $T_1 = T_{10,000} = 10^9$. Argue that the *speedup* of a randomized parallel algorithm should be defined as $E[T_1]/E[T_P]$, rather than $E[T_1/T_P]$.
- c. Argue that the *parallelism* of a randomized task-parallel algorithm should be defined as the ratio $E[T_1]/E[T_\infty]$.
- d. Parallelize the RANDOMIZED-QUICKSORT algorithm on page 192 by using recursive spawning to produce P-RANDOMIZED-QUICKSORT. (Do not parallelize RANDOMIZED-PARTITION.)
- e. Analyze your parallel algorithm for randomized quicksort. (*Hint*: Review the analysis of RANDOMIZED-SELECT on page 230.)
- f. Parallelize RANDOMIZED-SELECT on page 230. Make your implementation as parallel as possible. Analyze your algorithm. (*Hint*: Use the partitioning algorithm from Exercise 26.3-3.)

Chapter notes

Parallel computers and algorithmic models for parallel programming have been around in various forms for years. Prior editions of this book included material on sorting networks and the PRAM (Parallel Random-Access Machine) model. The data-parallel model [58, 217] is another popular algorithmic programming model, which features operations on vectors and matrices as primitives. The notion of sequential consistency is due to Lamport [275].

Graham [197] and Brent [71] showed that there exist schedulers achieving the bound of Theorem 26.1. Eager, Zahorjan, and Lazowska [129] showed that

any greedy scheduler achieves this bound and proposed the methodology of using work and span (although not by those names) to analyze parallel algorithms. Blelloch [57] developed an algorithmic programming model based on work and span (which he called “depth”) for data-parallel programming. Blumofe and Leiserson [63] gave a distributed scheduling algorithm for task-parallel computations based on randomized “work-stealing” and showed that it achieves the bound $E[T_P] \leq T_1/P + O(T_\infty)$. Arora, Blumofe, and Plaxton [20] and Blelloch, Gibbons, and Matias [61] also provided provably good algorithms for scheduling task-parallel computations. The recent literature contains many algorithms and strategies for scheduling parallel programs.

The parallel pseudocode and programming model were influenced by Cilk [290, 291, 383, 396]. The open-source project OpenCilk (www.opencilk.org) provides Cilk programming as an extension to the C and C++ programming languages. All of the parallel algorithms in this chapter can be coded straightforwardly in Cilk.

Concerns about nondeterministic parallel programs were expressed by Lee [281] and Bocchino, Adve, Adve, and Snir [64]. The algorithms literature contains many algorithmic strategies (see, for example, [60, 85, 118, 140, 160, 282, 283, 412, 461]) for detecting races and extending the fork-join model to avoid or safely embrace various kinds of nondeterminism. Blelloch, Fineman, Gibbons, and Shun [59] showed that deterministic parallel algorithms can often be as fast as, or even faster than, their nondeterministic counterparts.

Several of the parallel algorithms in this chapter appeared in unpublished lecture notes by C. E. Leiserson and H. Prokop and were originally implemented in Cilk. The parallel merge-sorting algorithm was inspired by an algorithm due to Akl [12].

Most problems described in this book have assumed that the entire input was available before the algorithm executes. In many situations, however, the input becomes available not in advance, but only as the algorithm executes. This idea was implicit in much of the discussion of data structures in Part III. The reason that you want to design, for example, a data structure that can handle n INSERT, DELETE, and SEARCH operations in $O(\lg n)$ time per operation is most likely because you are going to receive n such operation requests without knowing in advance what operations will be coming. This idea was also implicit in amortized analysis in Chapter 16, where we saw how to maintain a table that can grow or shrink in response to a sequence of insertion and deletion operations, yet with a constant amortized cost per operation.

An *online algorithm* receives its input progressively over time, rather than having the entire input available at the start, as in an *offline algorithm*. Online algorithms pertain to many situations in which information arrives gradually. A stock trader must make decisions today, without knowing what the prices will be tomorrow, yet wants to achieve good returns. A computer system must schedule arriving jobs without knowing what work will need to be done in the future. A store must decide when to order more inventory without knowing what the future demand will be. A driver for a ride-hailing service must decide whether to pick up a fare without knowing who will request rides in the future. In each of these situations, and many more, algorithmic decisions must be made without knowledge of the future.

There are several approaches for dealing with unknown future inputs. One approach is to form a probabilistic model of future inputs and design an algorithm that assumes future inputs conform to the model. This technique is common, for example, in the field of queuing theory, and it is also related to machine learning. Of course, you might not be able to develop a workable probabilistic model, or even if you can, some inputs might not conform to it. This chapter takes a differ-

ent approach. Instead of assuming anything about the future input, we employ a conservative strategy of limiting how poor a solution any input can entail.

This chapter, therefore, adopts a worst-case approach, designing online algorithms that guarantee the quality of the solution for all possible future inputs. We'll analyze online algorithms by comparing the solution produced by the online algorithm with a solution produced by an optimal algorithm that knows the future inputs, and taking a worst-case ratio over all possible instances. We call this methodology *competitive analysis*. We'll use a similar approach when we study approximation algorithms in Chapter 35, where we'll compare the solution returned by an algorithm that might be suboptimal with the value of the optimal solution, and determine a worst-case ratio over all possible instances.

We start with a “toy” problem: deciding between whether to take the elevator or the stairs. This problem will introduce the basic methodology of thinking about online algorithms and how to analyze them via competitive analysis. We will then look at two problems that use competitive analysis. The first is how to maintain a search list so that the access time is not too large, and the second is about strategies for deciding which cache blocks to evict from a cache or other kind of fast computer memory.

27.1 Waiting for an elevator

Our first example of an online algorithm models a problem that you likely have encountered yourself: whether you should wait for an elevator to arrive or just take the stairs. Suppose that you enter a building and wish to visit an office that is k floors up. You have two choices: walk up the stairs or take the elevator. Let's assume, for convenience, that you can climb the stairs at the rate of one floor per minute. The elevator travels much faster than you can climb the stairs: it can ascend all k floors in just one minute. Your dilemma is that you do not know how long it will take for the elevator to arrive at the ground floor and pick you up. Should you take the elevator or the stairs? How do you decide?

Let's analyze the problem. Taking the stairs takes k minutes, no matter what. Suppose you know that the elevator takes at most $B - 1$ minutes to arrive for some value of B that is considerably higher than k . (The elevator could be going up when you call for it and then stop at several floors on its way down.) To keep things simple, let's also assume that the number of minutes for the elevator to arrive is an integer. Therefore, waiting for the elevator and taking it k floors up takes anywhere from one minute (if the elevator is already at the ground floor) to $(B - 1) + 1 = B$ minutes (the worst case). Although you know B and k , you don't know how long the elevator will take to arrive this time. You can use competitive

analysis to inform your decision regarding whether to take the stairs or elevator. In the spirit of competitive analysis, you want to be sure that, no matter what the future brings (i.e., how long the elevator takes to arrive), you will not wait much longer than a seer who knows when the elevator will arrive.

Let us first consider what the seer would do. If the seer knows that the elevator is going to arrive in at most $k - 1$ minutes, the seer waits for the elevator, and otherwise, the seer takes the stairs. Letting m denote the number of minutes it takes for the elevator to arrive at the ground floor, we can express the time that the seer spends as the function

$$t(m) = \begin{cases} m + 1 & \text{if } m \leq k - 1, \\ k & \text{if } m \geq k. \end{cases} \quad (27.1)$$

We typically evaluate online algorithms by their *competitive ratio*. Let $\mathcal{U}$ denote the set (universe) of all possible inputs, and consider some input $I \in \mathcal{U}$. For a minimization problem, such as the stairs-versus-elevator problem, if an online algorithm A produces a solution with value $A(I)$ on input I and the solution from an algorithm F that knows the future has value $F(I)$ on the same input, then the competitive ratio of algorithm A is

$$\max \{A(I)/F(I) : I \in \mathcal{U}\}.$$

If an online algorithm has a competitive ratio of c , we say that it is *c-competitive*. The competitive ratio is always at least 1, so that we want an online algorithm with a competitive ratio as close to 1 as possible.

In the stairs-versus-elevator problem, the only input is the time for the elevator to arrive. Algorithm F knows this information, but an online algorithm has to make a decision without knowing when the elevator will arrive. Consider the algorithm “always take the stairs,” which always takes exactly k minutes. Using equation (27.1), the competitive ratio is

$$\max \{k/t(m) : 0 \leq m \leq B - 1\}. \quad (27.2)$$

Enumerating the terms in equation (27.2) gives the competitive ratio as

$$\max \left\{ \frac{k}{1}, \frac{k}{2}, \frac{k}{3}, \dots, \frac{k}{(k-1)}, \frac{k}{k}, \frac{k}{k}, \dots, \frac{k}{k} \right\} = k,$$

so that the competitive ratio is k . The maximum is achieved when the elevator arrives immediately. In this case, taking the stairs requires k minutes, but the optimal solution takes just 1 minute.

Now let’s consider the opposite approach: “always take the elevator.” If it takes m minutes for the elevator to arrive at the ground floor, then this algorithm will always take $m + 1$ minutes. Thus the competitive ratio becomes

$$\max \{(m + 1)/t(m) : 0 \leq m \leq B - 1\},$$

which we can again enumerate as

$$\max \left\{ \frac{1}{1}, \frac{2}{2}, \dots, \frac{k}{k}, \frac{k+1}{k}, \frac{k+2}{k}, \dots, \frac{B}{k} \right\} = \frac{B}{k}.$$

Now the maximum is achieved when the elevator takes $B - 1$ minutes to arrive, compared with the optimal approach of taking the stairs, which requires k minutes.

Hence, the algorithm “always take the stairs” has competitive ratio k , and the algorithm “always take the elevator” has competitive ratio B/k . Because we prefer the algorithm with smaller competitive ratio, if $k = 10$ and $B = 300$, we prefer “always take the stairs,” with competitive ratio 10, over “always take the elevator,” with competitive ratio 30. Taking the stairs is not always better, or necessarily more often better. It’s just that taking the stairs guards better against the worst-case future.

These two approaches of always taking the stairs and always taking the elevator are extreme solutions, however. Instead, you can “hedge your bets” and guard even better against a worst-case future. In particular, you can wait for the elevator for a while, and then if it doesn’t arrive, take the stairs. How long is “a while”? Let’s say that “a while” is k minutes. Then the time $h(m)$ required by this hedging strategy, as a function of the number m of minutes before the elevator arrives, is

$$h(m) = \begin{cases} m + 1 & \text{if } m \leq k, \\ 2k & \text{if } m > k. \end{cases}$$

In the second case, $h(m) = 2k$ because you wait for k minutes and then climb the stairs for k minutes. The competitive ratio is now

$$\max \{h(m)/t(m) : 0 \leq m \leq B - 1\}.$$

Enumerating this ratio yields

$$\max \left\{ \frac{1}{1}, \frac{2}{2}, \dots, \frac{k}{k}, \frac{k+1}{k}, \frac{2k}{k}, \frac{2k}{k}, \frac{2k}{k}, \dots, \frac{2k}{k} \right\} = 2.$$

The competitive ratio is now *independent* of k and B .

This example illustrates a common philosophy in online algorithms: we want an algorithm that guards against any possible worst case. Initially, waiting for the elevator guards against the case when the elevator arrives quickly, but eventually switching to the stairs guards against the case when the elevator takes a long time to arrive.

Exercises

27.1-1

Suppose that when hedging your bets, you wait for p minutes, instead of for k minutes, before taking the stairs. What is the competitive ratio as a function of p and k ? How should you choose p to minimize the competitive ratio?

27.1-2

Imagine that you decide to take up downhill skiing. Suppose that a pair of skis costs r dollars to rent for a day and b dollars to buy, where $b > r$. If you knew in advance how many days you would ever ski, your decision whether to rent or buy would be easy. If you'll ski for at least $\lceil b/r \rceil$ days, then you should buy skis, and otherwise you should rent. This strategy minimizes the total that you ever spend. In reality, you don't know in advance how many days you'll eventually ski. Even after you have skied several times, you still don't know how many more times you'll ever ski. Yet you don't want to waste your money. Give and analyze an algorithm that has a competitive ratio of 2, that is, an algorithm guaranteeing that, no matter how many times you ski, you never spend more than twice what you would have spent if you knew from the outset how many times you'll ski.

27.1-3

In "concentration solitaire," a game for one person, you have n pairs of matching cards. The backs of the cards are all the same, but the fronts contain pictures of animals. One pair has pictures of aardvarks, one pair has pictures of bears, one pair has pictures of camels, and so on. At the start of the game, the cards are all placed face down. In each round, you can turn two cards face up to reveal their pictures. If the pictures match, then you remove that pair from the game. If they don't match, then you turn both of them over, hiding their pictures once again. The game ends when you have removed all n pairs, and your score is how many rounds you needed to do so. Suppose that you can remember the picture on every card that you have seen. Give an algorithm to play concentration solitaire that has a competitive ratio of 2.

27.2 Maintaining a search list

The next example of an online algorithm pertains to maintaining the order of elements in a linked list, as in Section 10.2. This problem often arises in practice for hash tables when collisions are resolved by chaining (see Section 11.2), since each slot contains a linked list. Reordering the linked list of elements in each slot of the hash table can boost the performance of searches measurably.

The list-maintenance problem can be set up as follows. You are given a list L of n elements $\{x_1, x_2, \dots, x_n\}$. We'll assume that the list is doubly linked, although the algorithms and analysis work just as well for singly linked lists. Denote the position of element x_i in the list L by $r_L(x_i)$, where $1 \leq r_L(x_i) \leq n$. Calling $\text{LIST-SEARCH}(L, x_i)$ on page 260 thus takes $\Theta(r_L(x_i))$ time.

If you know in advance something about the distribution of search requests, then it makes sense to arrange the list ahead of time to put the more frequently searched elements closer to the front, which minimizes the total cost (see Exercise 27.2-1). If instead you don't know anything about the search sequence, then no matter how you arrange the list, it is possible that every search is for whatever element appears at the tail of the list. The total searching time would then be $\Theta(nm)$, where m is the number of searches.

If you notice patterns in the access sequence or you observe differences in the frequencies in which elements are accessed, then you might want to rearrange the list as you perform searches. For example, if you discover that every search is for a particular element, you could move that element to the front of the list. In general, you could rearrange the list after each call to LIST-SEARCH . But how would you do so without knowing the future? After all, no matter how you move elements around, every search could be for the last element.

But it turns out that some search sequences are “easier” than others. Rather than just evaluate performance on the worst-case sequence, let's compare a reorganization scheme with whatever an optimal offline algorithm would do if it knew the search sequence in advance. That way, if the sequence is fundamentally hard, the optimal offline algorithm will also find it hard, but if the sequence is easy, we can hope to do reasonably well.

To ease analysis, we'll drop the asymptotic notation and say that the cost is just i to search for the i th element in the list. Let's also assume that the only way to reorder the elements in the list is by swapping two adjacent elements in the list. Because the list is doubly linked, each swap incurs a cost of 1. Thus, for example, a search for the sixth element followed by moving it forward two places (entailing two swaps) incurs a total cost 8. The goal is to minimize the total cost of calls to LIST-SEARCH plus the total number of swaps performed.

The online algorithm that we'll explore is $\text{MOVE-TO-FRONT}(L, x)$. This procedure first searches for x in the doubly linked list L , and then it moves x to the front of the list.¹ If x is located at position $r = r_L(x)$ before the call, MOVE-TO-FRONT swaps x with the element in position $r - 1$, then with the element in position $r - 2$,

¹ The path-compression heuristic in Section 19.3 resembles MOVE-TO-FRONT , although it would be more accurately expressed as “move-to-next-to-front.” Unlike MOVE-TO-FRONT in a doubly linked list, path compression can relocate multiple elements to become “next-to-front.”

element searched	FORESEE					MOVE-TO-FRONT				
	L	search cost	swap cost	search + swap cost	cumulative cost	L	search cost	swap cost	search + swap cost	cumulative cost
5	$\langle 1, 2, 3, 4, 5 \rangle$	5	0	5	5	$\langle 1, 2, 3, 4, 5 \rangle$	5	4	9	9
3	$\langle 1, 2, 3, 4, 5 \rangle$	3	3	6	11	$\langle 5, 1, 2, 3, 4 \rangle$	4	3	7	16
4	$\langle 4, 1, 2, 3, 5 \rangle$	1	0	1	12	$\langle 3, 5, 1, 2, 4 \rangle$	5	4	9	25
4	$\langle 4, 1, 2, 3, 5 \rangle$	1	0	1	13	$\langle 4, 3, 5, 1, 2 \rangle$	1	0	1	26

Figure 27.1 The costs incurred by the procedures FORESEE and MOVE-TO-FRONT when searching for the elements 5, 3, 4, and 4, starting with the list $L = \langle 1, 2, 3, 4, 5 \rangle$. If FORESEE instead moved 3 to the front after the search for 5, the cumulative cost would not change, nor would the cumulative cost change if 4 moved to the second position after the search for 5.

and so on, until it finally swaps x with the element in position 1. Thus if the call $\text{MOVE-TO-FRONT}(L, 8)$ executes on the list $L = \langle 5, 3, 12, 4, 8, 9, 22 \rangle$, the list becomes $\langle 8, 5, 3, 12, 4, 9, 22 \rangle$. The call $\text{MOVE-TO-FRONT}(L, k)$ costs $2r_L(k) - 1$: it costs $r_L(k)$ to search for k , and it costs 1 for each of the $r_L(k) - 1$ swaps that move k to the front of the list.

We'll see that MOVE-TO-FRONT has a competitive ratio of 4. Let's think about what this means. MOVE-TO-FRONT performs a series of operations on a doubly linked list, accumulating cost. For comparison, suppose that there is an algorithm FORESEE that knows the future. Like MOVE-TO-FRONT, it also searches the list and moves elements around, but after each call it optimally rearranges the list for the future. (There may be more than one optimal order.) Thus FORESEE and MOVE-TO-FRONT maintain different lists of the same elements.

Consider the example shown in Figure 27.1. Starting with the list $\langle 1, 2, 3, 4, 5 \rangle$, four searches occur, for the elements 5, 3, 4, and 4. The hypothetical procedure FORESEE, after searching for 3, moves 4 to the front of the list, knowing that a search for 4 is imminent. It thus incurs a swap cost of 3 upon its second call, after which no further swap costs accrue. MOVE-TO-FRONT incurs swap costs in each step, moving the found element to the front. In this example, MOVE-TO-FRONT has a higher cost in each step, but that is not necessarily always the case.

The key to proving the competitive bound is to show that at any point, the total cost of MOVE-TO-FRONT is not much higher than that of FORESEE. Surprisingly, we can determine a bound on the costs incurred by MOVE-TO-FRONT relative to FORESEE even though MOVE-TO-FRONT cannot see the future.

If we compare any particular step, MOVE-TO-FRONT and FORESEE may be operating on very different lists and do very different things. If we focus on the search for 4 above, we observe that FORESEE actually moves it to the front of the list early, paying to move the element to the front before it is accessed. To capture this con-

cept, we use the idea of an *inversion*: a pair of elements, say a and b , in which a appears before b in one list, but b appears before a in another list. For two lists L and L' , let $I(L, L')$, called the *inversion count*, denote the number of inversions between the two lists, that is, the number of pairs of elements whose order differs in the two lists. For example, with lists $L = \langle 5, 3, 1, 4, 2 \rangle$ and $L' = \langle 3, 1, 2, 4, 5 \rangle$, then out of the $\binom{5}{2} = 10$ pairs, exactly five of them— $(1, 5)$, $(2, 4)$, $(2, 5)$, $(3, 5)$, $(4, 5)$ —are inversions, since these pairs, and only these pairs, appear in different orders in the two lists. Thus the inversion count is $I(L, L') = 5$.

In order to analyze the algorithm, we define the following notation. Let L_i^M be the list maintained by MOVE-TO-FRONT immediately after the i th search, and similarly, let L_i^F be FORESEE's list immediately after the i th search. Let c_i^M and c_i^F be the costs incurred by MOVE-TO-FRONT and FORESEE on their i th calls, respectively. We don't know how many swaps FORESEE performs in its i th call, but we'll denote that number by t_i . Therefore, if the i th operation is a search for element x , then

$$c_i^M = 2r_{L_{i-1}^M}(x) - 1, \quad (27.3)$$

$$c_i^F = r_{L_{i-1}^F}(x) + t_i. \quad (27.4)$$

In order to compare these costs more carefully, let's break down the elements into subsets, depending on their positions in the two lists before the i th search, relative to the element x being searched for in the i th search. We define three sets:

$$\begin{aligned} BB &= \{\text{elements before } x \text{ in both } L_{i-1}^M \text{ and } L_{i-1}^F\}, \\ BA &= \{\text{elements before } x \text{ in } L_{i-1}^M \text{ but after } x \text{ in } L_{i-1}^F\}, \\ AB &= \{\text{elements after } x \text{ in } L_{i-1}^M \text{ but before } x \text{ in } L_{i-1}^F\}. \end{aligned}$$

We can now relate the position of element x in L_{i-1}^F and L_{i-1}^M to the sizes of these sets:

$$r_{L_{i-1}^M}(x) = |BB| + |BA| + 1, \quad (27.5)$$

$$r_{L_{i-1}^F}(x) = |BB| + |AB| + 1. \quad (27.6)$$

When a swap occurs in one of the lists, it changes the relative positions of the two elements involved, which in turn changes the inversion count. Suppose that elements x and y are swapped in some list. Then the only possible difference in the inversion count between this list and *any* other list depends on whether (x, y) is an inversion. In fact, the inversion count of (x, y) with respect to any other list *must* change. If (x, y) is an inversion before the swap, it no longer is afterward, and vice versa. Therefore, if two consecutive elements x and y swap positions in a list L , then for any other list L' , the value of the inversion count $I(L, L')$ either increases by 1 or decreases by 1.